Youtube

Functional requirements:
user should be able to view/download any video based on his preference,
user should be able to search for any video based on his preference,
video resolution should get adjusted based on the network bandwidth

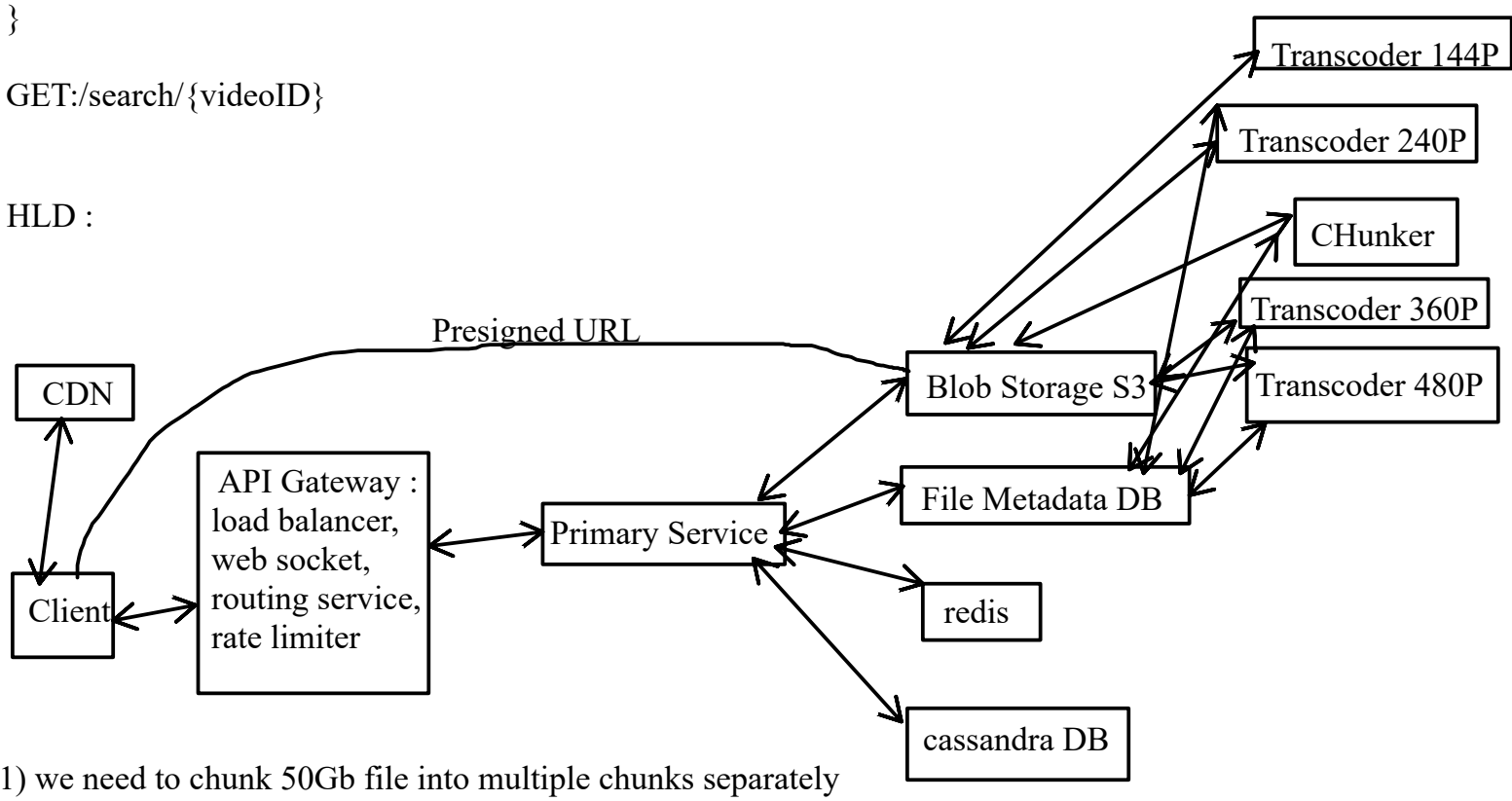
Non-functional requirements:
Data-partitioning ,
load balancing.
fault tolearance,
vertical scalability,
horizontal scalability,
low latency,
high throughput,
CAP theorem (here eventual consistency is fine , but here we need 99.99% uptime i.e Availability>>Consistency)

Core Entities:
user,
videos,
history,
filematadata

API :
POST:/upload/video
{
userId,
{filemetadata}
}

GET:/search/{videoID}

HLD :



1) we need to chunk 50Gb file into multiple chunks separately
2)we need to fingerprint or hash each piece of these chunks separately
3) we need to upload each piece of these chunks separately
4)update completed status for each piece of these chunks separately